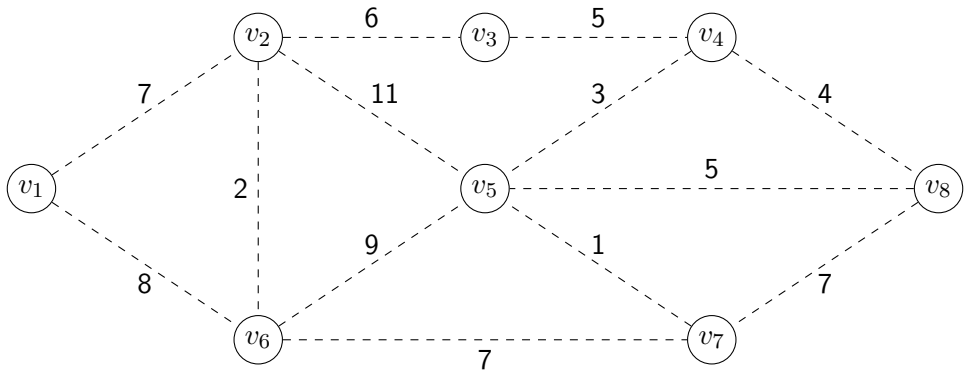
Definitions and introduction

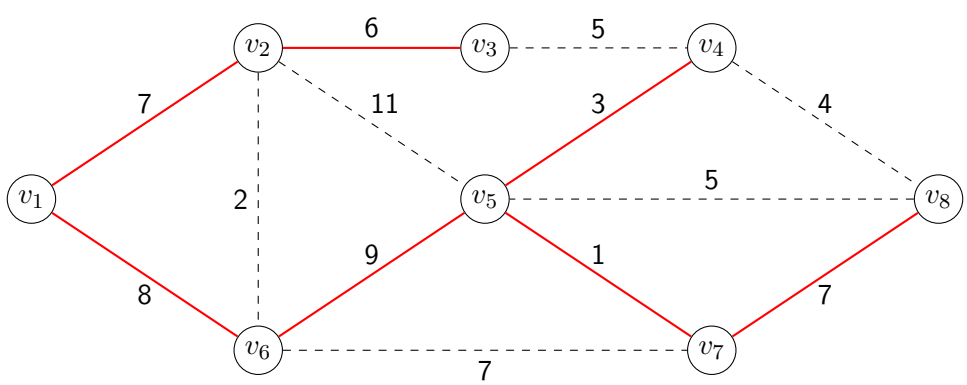
In previous courses we have discussed graph traversal algorithms – namely Breadth First Search and Depth First Search. One might recall that both of these algorithms find subtrees within the graph; this is common for graph algorithms and will continue for our next two algorithms. We shall first discuss the Minimum Spanning Tree; consider the undirected edge-weighted graph



Definition 1. A **spanning tree** of a graph G is a tree composed of all of the vertices and some of the edges of G. The **weight** of a spanning tree is the sum of all of the edge weights.

Definition 2. A **minimum weight spanning tree** is a subtree of a connected, edge-weighted undirected graph that connects all the vertices together with the minimum total edge weight.

Given these definitions, can you find a spanning tree in the graph? What is the weight? Can you find a tree with a smaller weight?
One spanning tree with the graph is seen below in red.

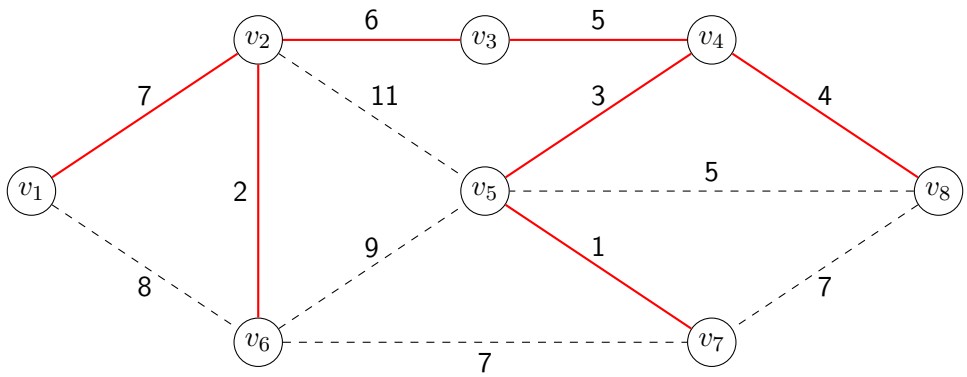


The weight of this spanning tree is

$$\text{weight}(T) = \sum_{e \in T.E} \text{weight}(e) = 7 + 8 + 6 + 9 + 3 + 1 + 7 = 38$$

Can we improve upon this?

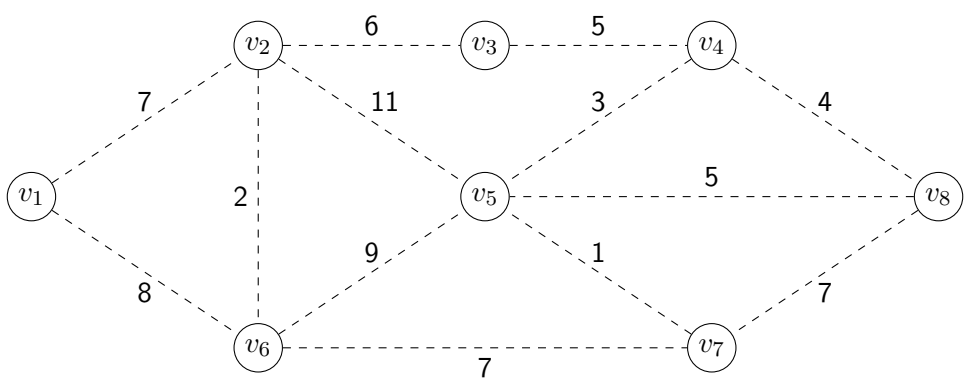
Here is an alternative spanning tree



and it has weight

$$\text{weight}(T) = \sum_{e \in T.E} \text{weight}(e) = 7 + 2 + 6 + 5 + 3 + 1 + 4 = 28$$

which is definitely less than our previous tree. Is this the minimum? How can we know with certainty that the spanning tree we have found is the solution? Until now we have mostly focused on analyzing the running time of our algorithms and haven't concerned ourselves overmuch with their correctness since it was usually "obvious" that our algorithm was correct. First, we shall devise an algorithm for solving this problem and then we shall prove that the algorithm does in fact successfully find *a* minimum spanning tree of the graph. Consider the graph given below,

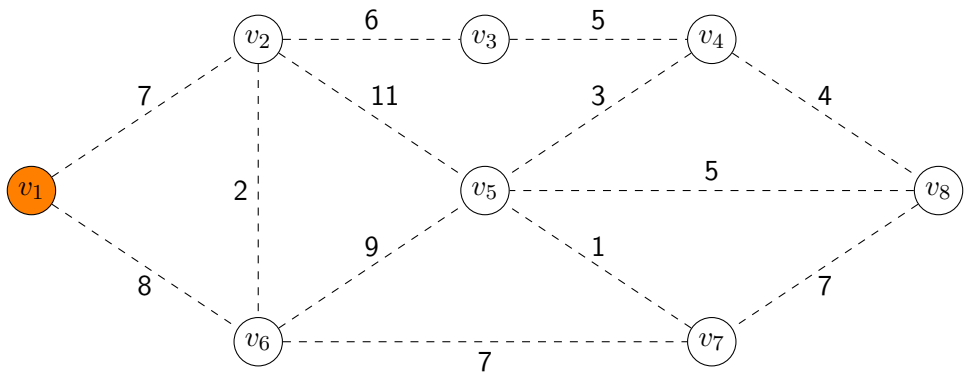


Let us choose a starting vertex for our tree of v_1. Our algorithm will behave as follows:

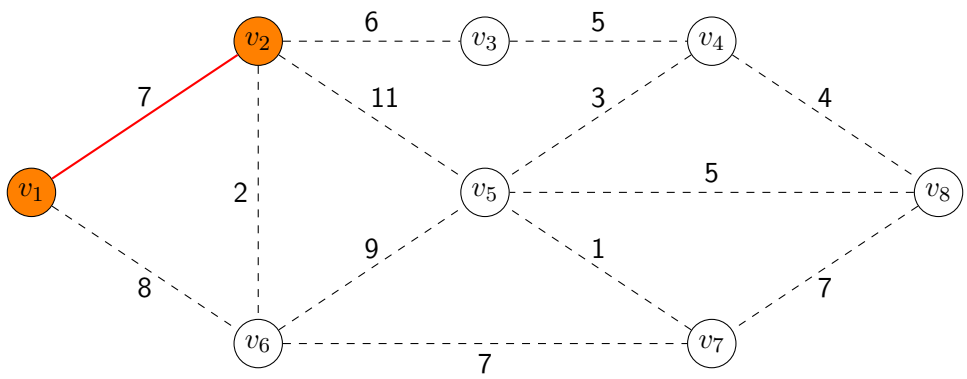
1. Maintain a tree of discovered nodes.
2. Add undiscovered nodes to the tree iteratively by selecting an undiscovered node that is adjacent to our tree of discovered nodes with the least edge weight.

This is a "greedy" algorithm; this means that it will always make the "best local decision available". In this case that means choosing the least weight edge to add to the graph with no potential "forethought" about whether adding a more expensive edge *right now* may result in better "options" in the future.

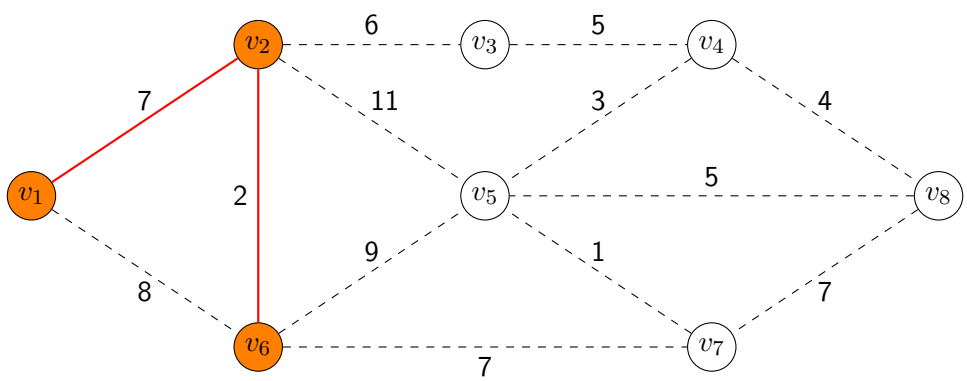
We shall color the "discovered nodes" in orange and the undiscovered nodes in white.



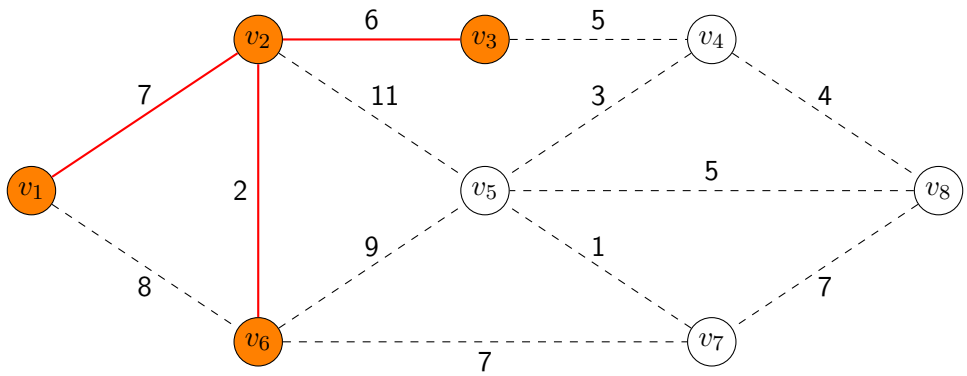
Given the current tree the best edge that connects undiscovered vertices (white) to discovered vertices (orange) is (v_1, v_2) which has an edge weight of 7. Therefore, we "discover" v_2 and add (v_1, v_2) to the spanning tree.



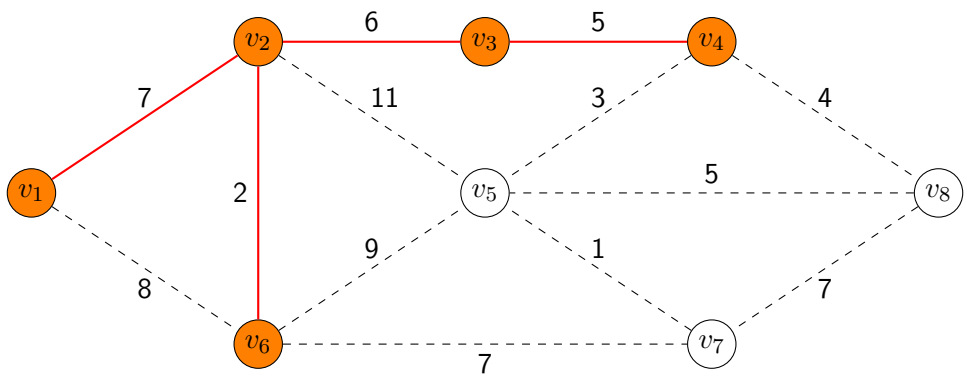
There are now 4 edges that connect undiscovered nodes to discovered nodes: (v_1, v_6), (v_2, v_3), (v_2, v_5), and (v_2, v_6). The edge (v_2, v_6) with weight 2 has the smallest edge weight among possible options so we add v_6 to the discovered nodes and (v_2, v_6) to the tree.



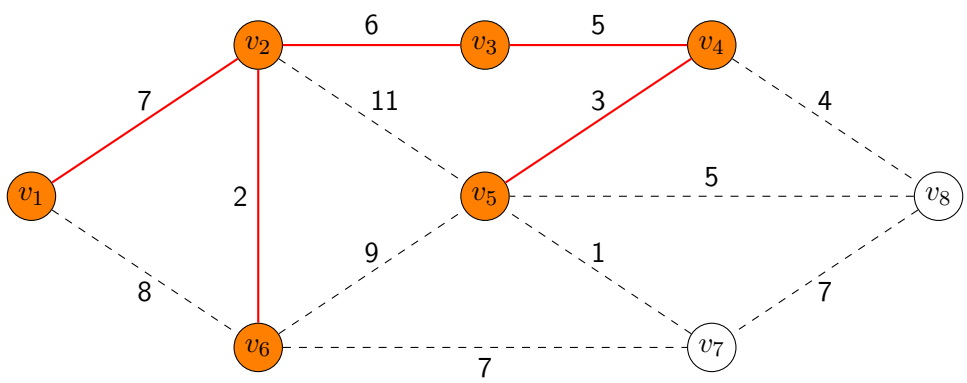
We shall now accelerate our process: the next edge that we add to the spanning tree is (v_2, v_3) with edge weight 6



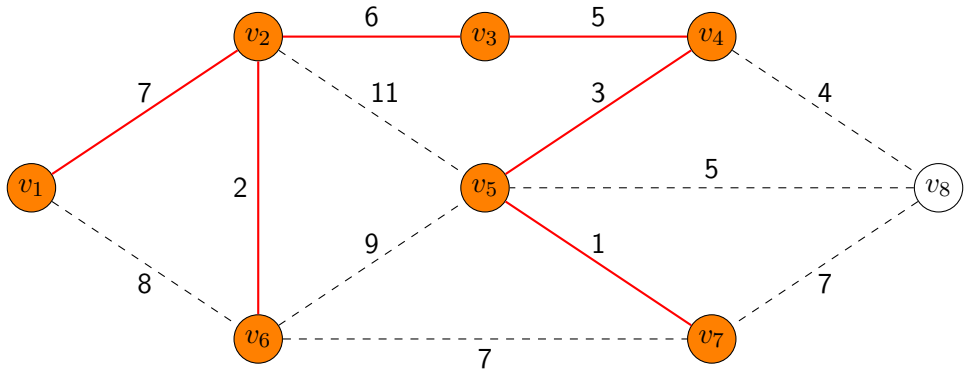
Next we add edge (v_3, v_4) with edge weight 5,



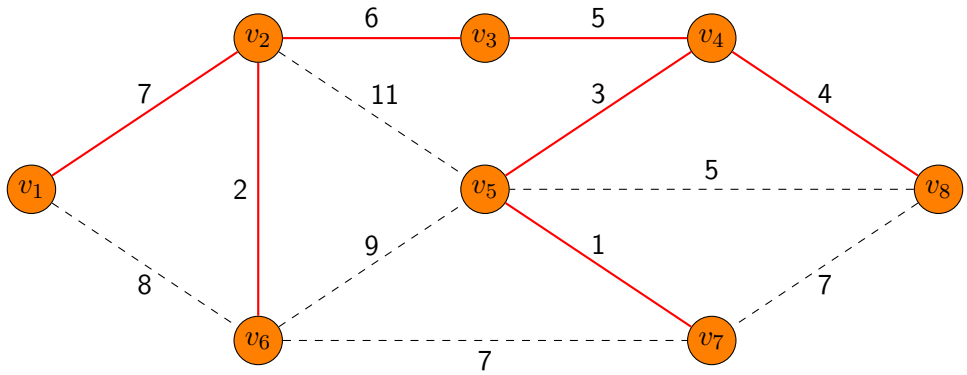
Next we add (v_4, v_5) with edge weight 3,



Next we add (v_5, v_7) with edge weight 1,



And finally we add (v_4, v_8) with edge weight 4,



The weight of the spanning tree discovered by this algorithm is

$$\text{weight}(T) = \sum_{e \in T.E} \text{weight}(e) = 7 + 2 + 6 + 5 + 3 + 1 + 4 = 28.$$

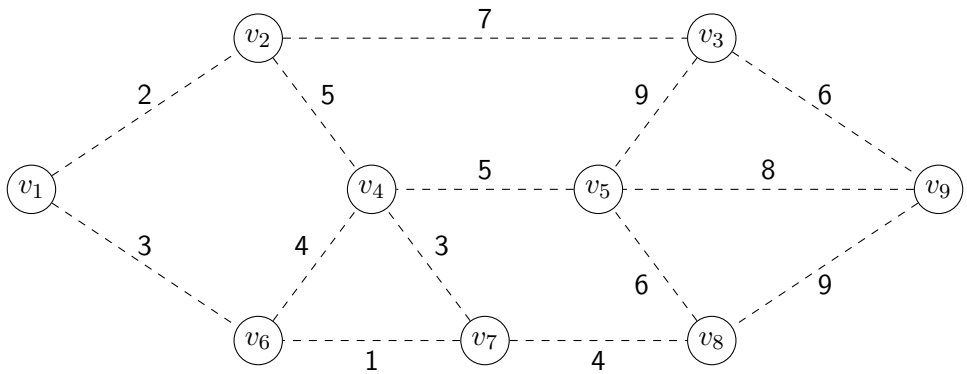
Now that we have formally devised our algorithm, we must formally prove that the answer achieved by this algorithm is in fact a minimum spanning tree. It can sometimes be useful to express the algorithm is pseudocode before analyzing the correctness.

Input: A weighted graph G

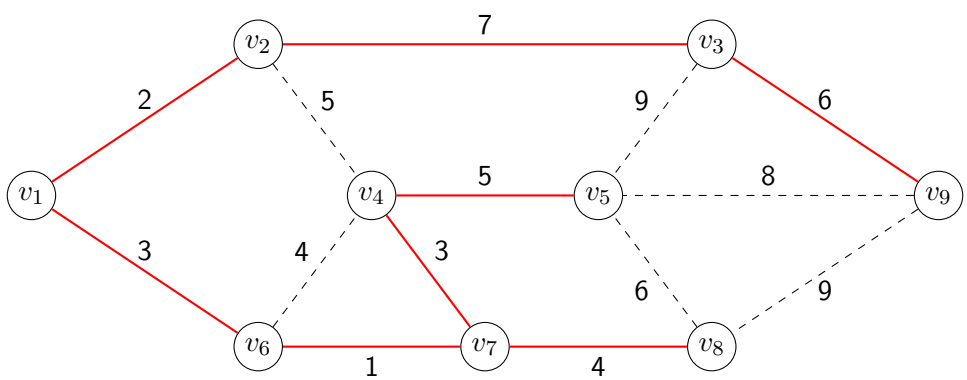
Output: Minimum spanning tree of G

```
function PrimMST(G)
    U = G.V − {v_1}
    v_1.MSTparent = NULL
    while (U ≠ ∅) and (∃ edge from (G.V − U) to U) do
        (v_i, v_j) = minimum weight edge from G.V − U to U
        v_j.MSTparent = v_i
        U = U − {v_j}
    end while
end function
```

We leave it to the reader to practice with this algorithm on the weighted undirected graph given below. Using the algorithm can you find a minimum spanning tree in the graph?



One minimum spanning tree in the graph is given below



We shall prove the algorithm's correctness in two steps. We have abstracted some results into the Minimum Spanning Tree Lemma; these results will be useful when we prove the Minimum Spanning Tree Theorem (most lemmas are not inherently useful results, but are instead useful building blocks for more important theorems).

Lemma 1 (Minimum Spanning Tree Lemma)**.** Let A be a spanning tree of graph G and let e_B be an edge of G which is not in A.

1. $A \cup \{e_B\}$ contains a cycle C.
2. If $e_A \neq e_B$ is an edge of cycle C, then $B = (A \cup \{e_B\}) - \{e_A\}$ is a spanning tree of G.
3. If weight$(e_B) \leq$ weigth(e_A), then weight$(B) \leq$ weigth(A)

Proof.

1. In A there was already a path $p = (v_1, v_2, \ldots, v_k)$ connecting the vertices incident on e_B since a spanning tree is connected. Therefore, $p \cup e_B$ forms a cycle.
2. Since A was a tree it was acyclic, therefore by removing e_A we destroy the cycle. Alternatively one could argue that B is a connected graph with $|V| - 1$ edges and therefore must be a tree.
3. Trivial. But if one wanted to show it, let $A.E = \{e_1, e_2, \ldots, e_{|V|-2}, e_A\}$ be the edges of A, then

$$\begin{aligned}
\text{weight}(B) &= \text{weight}(e_1) + \text{weight}(e_2) + \cdots + \text{weight}(e_{|V|-2}) + \text{weight}(e_B) && (1)\\
&\leq \text{weight}(e_1) + \text{weight}(e_2) + \cdots + \text{weight}(e_{|V|-2}) + \text{weight}(e_A) && (2)\\
&= \text{weight}(A) && (3)
\end{aligned}$$

□

With this lemma at our fingertips we are now ready to introduce and tackle the theorem,

Theorem 1 (Minimum Spanning Tree Theorem)**.** Let T be a subtree of a minimum spanning tree. If e is a minimum weight edge connecting T to some vertex not in T, then $T \cup \{e\}$ is a subtree of a minimum spanning tree.

Proof. Let T be a subtree of some minimum spanning tree A of G and let e by a minimum weight edge connecting T to some vertex not in T.
If e is in A, then clearly $T \cup \{e\}$ is a subtree of A.
If e is not in A, then $A \cup \{e\}$ contains a cycle C by the MST Lemma 1. Some edge $e_A \neq e$ of C must be an edge from T to a vertex not in T since e connects T to a vertex not in T. Let $B = (A \cup \{e\}) - \{e_A\}$; note that B is a spanning tree by the MST Lemma 2. Since e is a minimum weight from T to vertices not in T, weight$(e) \leq$ weigth(e_A). Thus, weight$(B) \leq$ weigth(A) by MST Lemma 3. Since A is a minimum spanning tree, weight$(B) =$ weigth(A) and therefore B is a minimum spanning tree of G containing $T \cup \{e\}$. □

Note that this theorem states that iteratively applying our greedy decision making process ensures that we find a minimum spanning tree since the graph $A = (\{v_1\}, \varnothing)$ is a tree.

Armed with the knowledge of the correctness of algorithm, we shall attempt to analyze the running time of our algorithm.

Input: A weighted graph G

Output: Minimum spanning tree of G

```
1: function PrimMST(G)
2:     U = G.V − {v_1}
3:     v_1.MSTparent = NULL
4:     while (U ≠ ∅) and (∃ edge from (G.V − U) to U) do
5:         (v_i, v_j) = minimum weight edge from G.V − U to U
6:         v_j.MSTparent = v_i
7:         U = U − {v_j}
8:     end while
9: end function
```

Let $n =$ number of verticies of G and $m =$ number of edges of G. We shall assume that G is a connected graph to simplify our analysis.

Line 2 of the code takes $\Theta(n)$ time to create the set U of size $n-1$.

Line 3 of the code takes $\Theta(1)$ time.

Since G is a connected graph, the while loop on lines 4-8 will run approximately n times to add each vertex to the spanning tree. We note that line 4 may not take constant time to execute since we may not be able to determine whether an edge exists from $G.V - U$ to U in constant time. In the worst case this could take cm time since we may have to check every edge of the graph.

The seemingly innocuous line 5 may not take constant time since in order to determine the minimum weight edge from $G.V - U$ we may have to examine every edge in the graph (or at the very least every edge in U or $G.V - U$). For example,

```
min = ∞
minEdge = NULL
for each edge (u, v) ∈ G.E do
    if u ∈ U and v ∈ G.V − U and weight(u, v) ≤ min then
        min = weight(u, v)
        minEdge = (u, v)
    end if
end for
```

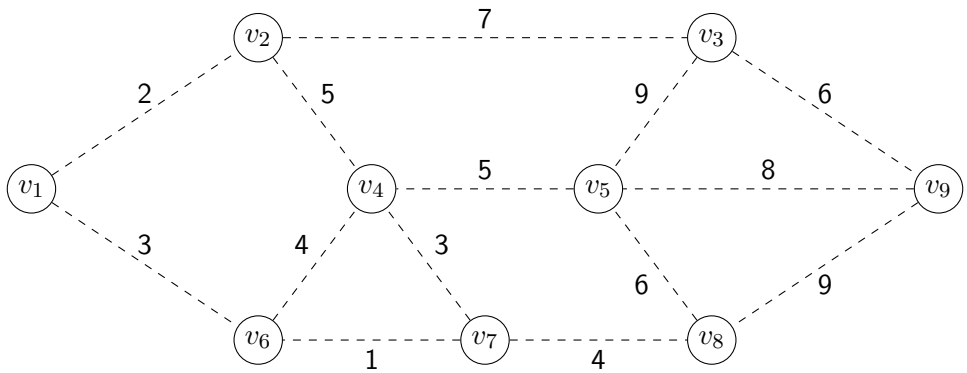
would find the minimum weight edge from U to $G.V - U$ with $O(1)$ spacial complexity. Therefore, we shall assume that line 5 takes $\Theta(m)$ time.

Lines 6 and 7 both take $\Theta(1)$ time.

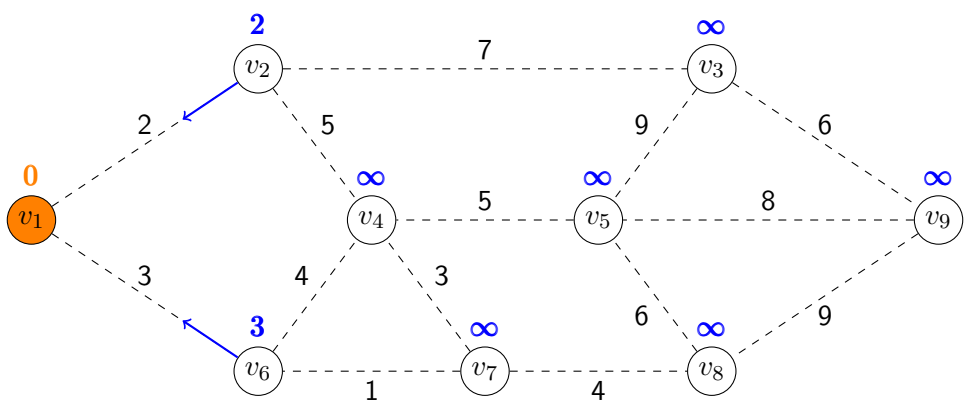
Therefore, the running time of `PrimsMST` is in $\boxed{\Theta(mn)}$. We note that $n - 1 \leq m \leq \frac{n(n-1)}{2}$ for a connected graph, so `PrimsMST` is in $\Omega(n^2)$ and $O(n^3)$, which are both rather unfortunate bounds.

Can we improve upon the running time of this algorithm? One idea is try and increase the storage of the algorithm to reduce the running time.

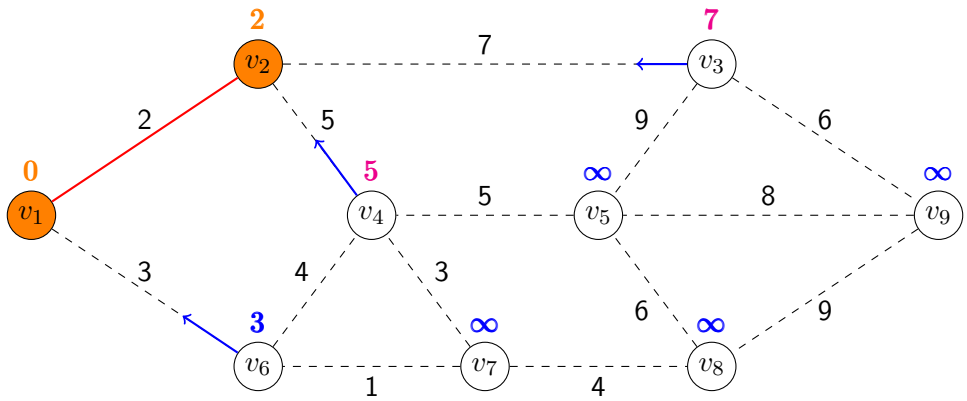
Here is our first idea: what if we store the "best edge to vertex" that we have found so far? That is, each vertex that is currently undiscovered will store the minimum weight edge that connects to the discovered nodes. Whenever we add a new discovered node, we update these edges as appropriate, then select the vertex with the minimum edge weight stored at that vertex. Let us demonstrate this algorithm; given the graph below



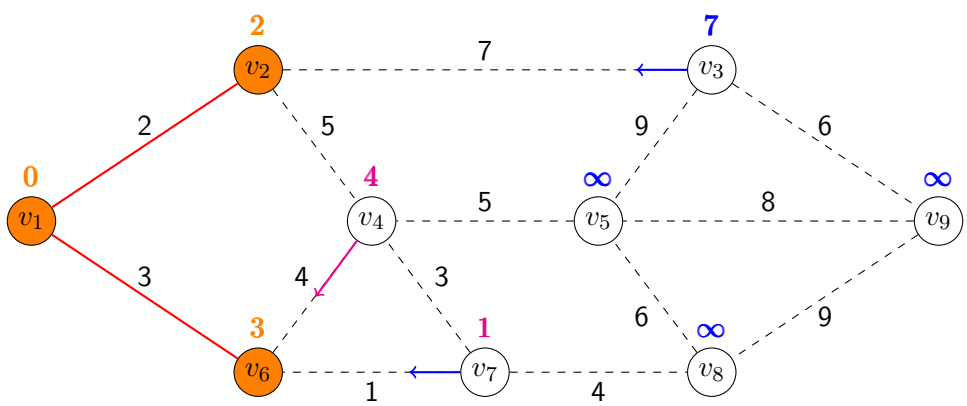
suppose that we begin our minimum spanning tree at vertex v_1, then we initialize the weight of every vertex and the denote the edge with that weight



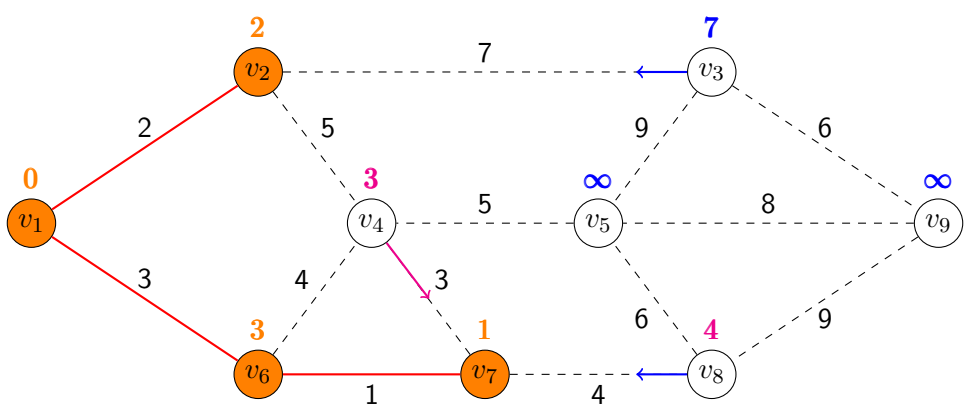
we then iterate over the undiscovered vertices, find the vertex with minimum edge weight, and add that vertex and edge to the minimum spanning tree. In this case we add vertex v_2 and edge (v_1, v_2). Once we add this edge, we check every edge incident on v_2 and update the weight of the vertices adjacent to v_2 if we find "a better edge". Doing so the graph becomes



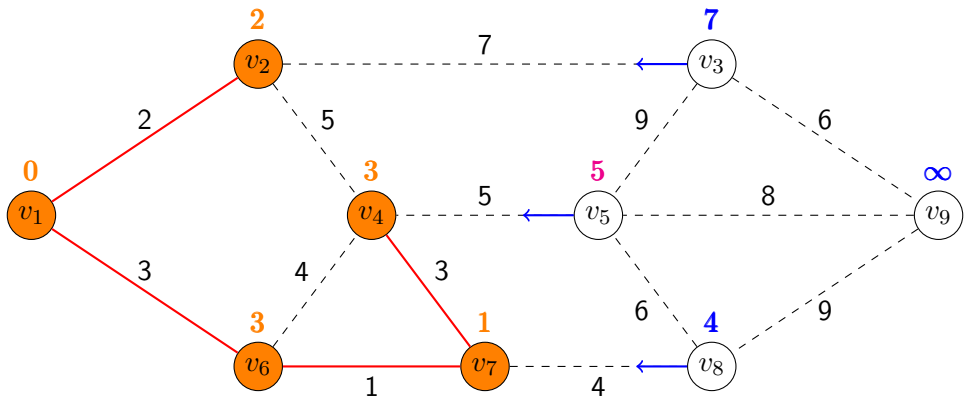
We shall now repeat this process. Among the undiscovered nodes the smallest edge weight found thus far is 3, therefore we add (v_1, v_6) and v_6 to the minimum spanning tree.



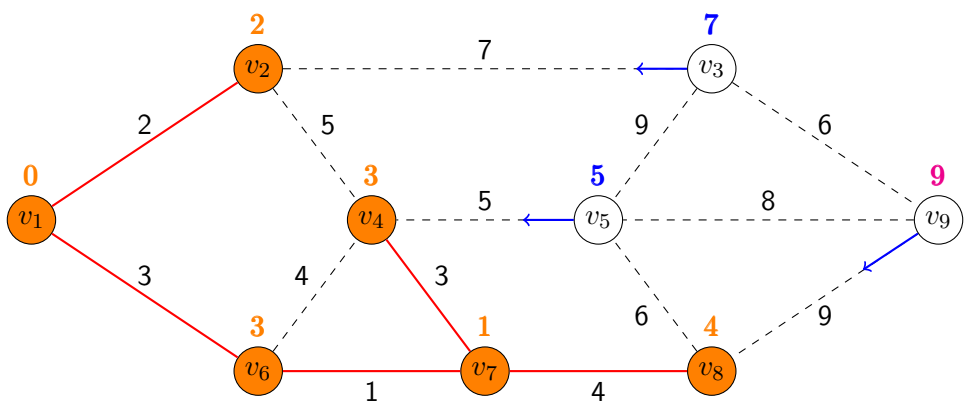
Among the undiscovered nodes the smallest edge weight found thus far is 1, therefore we add (v_6, v_7) and v_7 to the minimum spanning tree.



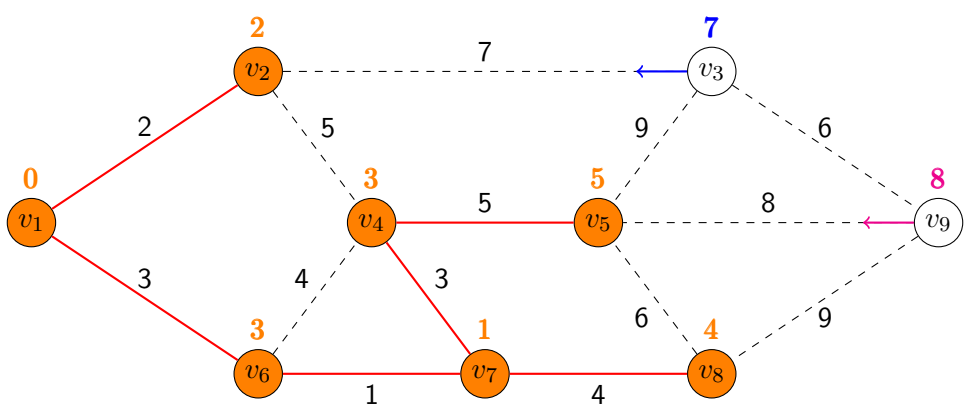
Among the undiscovered nodes the smallest edge weight found thus far is 3, therefore we add (v_7, v_4) and v_4 to the minimum spanning tree.



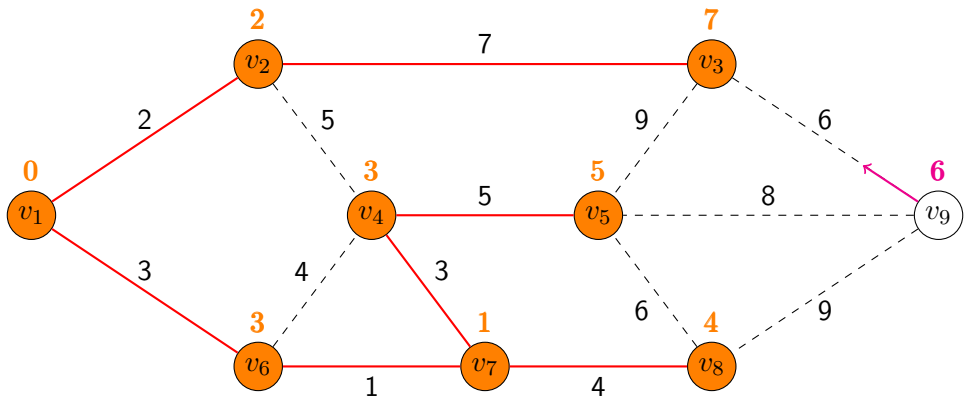
Among the undiscovered nodes the smallest edge weight found thus far is 4, therefore we add (v_7, v_8) and v_8 to the minimum spanning tree.



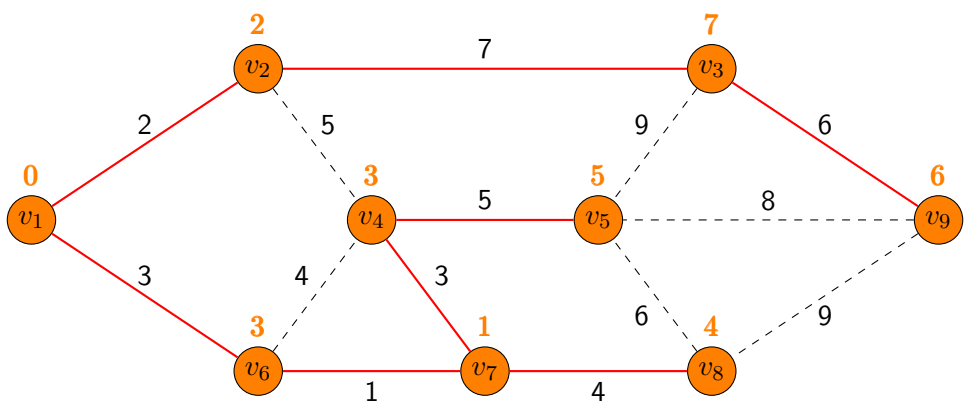
Among the undiscovered nodes the smallest edge weight found thus far is 5, therefore we add (v_4, v_5) and v_5 to the minimum spanning tree.



Among the undiscovered nodes the smallest edge weight found thus far is 7, therefore we add (v_2, v_3) and v_3 to the minimum spanning tree.



Among the undiscovered nodes the smallest edge weight found thus far is 6, therefore we add (v_3, v_9) and v_9 to the minimum spanning tree.



You may have noticed that as we went through the process that we needed to iterate only over $U - G.V$ and that we need to only update the cost at a very small number of nodes. These two benefits shall pay dividends for our algorithm.

Consider our modified version of Prim's Minimum Spanning Tree algorithm, where each graph node has been modified to have two fields `cost` which contains the cost of adding the node to the minimum spanning tree and `parent` that contains the parent corresponding to `cost`.

Input: A weighted graph G

Output: Minimum spanning tree of G

```
1: function PrimMST(G)
2:     U = G.V
3:     for each vertex v_i ∈ G.V − {v_1} do
4:         v_i.cost = ∞
5:     end for
6:     v_1.cost = 0
7:     v_1.parent = NULL
8:     while (U ≠ ∅) and (v_i.cost < ∞ for some v_i ∈ U) do
9:         v_j = v_i ∈ U with minimum v_i.cost
10:        U = U − {v_j}
11:        v_j.MSTparent = v_j.parent
12:        for each edge (v_j, v_k) incident on v_j do
13:            if (v_k ∈ U) and (weight(v_j, v_k) < v_k.cost then
14:                v_k.parent = v_j
15:                v_k.cost = weight(v_j, v_k)
16:            end if
17:        end for
18:    end while
19: end function
```

Let $n =$ number of verticies of G and $m =$ number of edges of G. We shall assume that G is a connected graph to simplify our analysis.

Lines 3-5 take $\Theta(n)$ time since we must iterate over $n-1$ vertices.

Lines 6 and 7 take $\Theta(1)$ time.

The while loop on lines 8-18 iterates n times.

Line 9 must iterate over every vertex in U to determine the minimum and therefore takes $\Theta(|U|)$ time. Notice that as the while loop iterates $|U|$ decreases from n down to 1. Therefore, the cost line 9 on the ith iteration is $c(n-i+1)$. Therefore, the total cost of line 9 is

$$\text{Cost of line 9} = \sum_{i=1}^{n} c(n-i+1) = \sum_{i=1}^{n} i = \frac{n(n+1)}{2}$$

Lines 10 and 11 take $\Theta(1)$ time.

The for loop on lines 12-17 iterates $\deg(v_j)$ times and each iteration takes constant time (note that

determining whether $v_k \in U$ in constant time may require some cautious programming). Therefore,

$$\text{Cost of lines 12-17} = \sum_{j=1}^{n} c\text{deg}(v_j) = 2cm$$

Therefore, the total cost of the while loop on lines 8-18 is $\Theta(n^2 + m)$. Furthermore, the running time of the algorithm is

$$\boxed{T(m, n) \in \Theta(n^2 + m)}$$

which is demonstrably better than the running time of our previous implementation. We achieved this superior running time by increasing the storage requirements of the algorithm.
Can we further improve this running time? We notice that primary operations occurring within the algorithm are

1. Inserting elements.
2. Extracting the minimum value.
3. Updating the cost of the nodes.

Conveniently, we have a data structure that is *expressly built* to perform these operations efficiently – the Min Heap! Therefore, we shall attempt to modify the algorithm to take advantage of this data structure.

A quick refresher on a Minimum Priority Queue implemented using a Min Heap,

Method	**Functionality**	**Runtime**
`Q.Init()`	Initialize the minimum priority queue	$\Theta(1)$
`Q.Insert`(Object x, Key k)	Inserts x into Min. PQ with key k	$\Theta(\log(s))$
`Q.ExtractMin()`	Extracts element with minimum key	$\Theta(\log(s))$
`Q.DecreaseKey`(Object x, Key k)	Decrease the key value associated with x to k	$\Theta(\log(s))$
`Q.IsEmpty()`	Returns true if Min. PQ is empty.	$\Theta(1)$
`Q.IsNotEmpty()`	Returns true if Min. PQ is not empty.	$\Theta(1)$
`Q.Contains`(Object x)	Returns true if Min. PQ contains x.	$\Theta(1)$
`Q.Key`(Object x)	Returns key value associated with x.	$\Theta(1)$
`Q.MinKey()`	Returns minimum key value.	$\Theta(1)$

Note: We will need to add an array of size n (the number of vertices in the graph) to the Minimum Priority Queue that indicates the location of each vertex in the heap to ensure that `Q.Contains(`x`)` runs in constant time.

The code for our third implementation of Prim's Minimum Spanning Tree algorithm is very similar to our second implementation, with the primary change being that we shall store the costs in Minimum Priority Queue. Assume that `Q` is a Minimum Priority Queue implemented as a Min Heap.

Input: A weighted graph G

Output: Minimum spanning tree of G

```
1: function PrimMST(G)
2:     Q.Init()
3:     for each vertex v_i ∈ G.V − {v_1} do
4:         Q.Insert(v_i, ∞)
5:     end for
6:     Q.Insert(v_1, 0)
7:     v_1.parent = NULL
8:     while (Q.IsNotEmpty()) and (Q.MinKey() ≠ ∞) do
9:         v_j = Q.ExtractMin()
10:        v_j.MSTparent = v_j.parent
11:        for each edge (v_j, v_k) incident on v_j do
12:            if (Q.Contains(v_k)) and (weight(v_j, v_k) < Q.Key(v_k)) then
13:                v_k.parent = v_j
14:                Q.DecreaseKey(v_k, weight(v_j, v_k))
15:            end if
16:        end for
17:    end while
18: end function
```

Let $n =$ number of verticies of G and $m =$ number of edges of G. We shall assume that G is a connected graph to simplify our analysis.

The running time of lines 3-7 is

$$\sum_{s=1}^{n} T_{in}(s) = \sum_{s=1}^{n} c\log(s) \in \Theta(n\log(n))$$

since the running time of `Q.Insert` is $\Theta(\log(s))$ where s is the size of the heap.

The while loop on lines 8-17 runs n times.

The size before extraction of the heap on line 9 starts at n and decreases down to 1, so

$$\text{Cost of line 9} = \sum_{s=1}^{n} T_{em}(s) = \sum_{s=1}^{n} c\log(s) \in \Theta(n\log(n))$$

The running time of line 10 is $\Theta(1)$.

The for loop on lines 11-16 runs $\deg(v_j)$ times and each iteration of the loop takes $\Theta(\log(s))$ time, where s is the number of elements in the heap.

Let s_j be the size of the heap when v_j is extracted from the heap, then

$$\text{Cost of lines 11-16} = \sum_{j=1}^{n} c\text{deg}(v_j)\log(s_j) \leq \sum_{s=1}^{n} c\text{deg}(v_j)c\log(n) = 2cm\log(n)$$

Bounding this summation from below proves too difficult for this course and requires more assumptions than we are willing to make here.
Therefore,

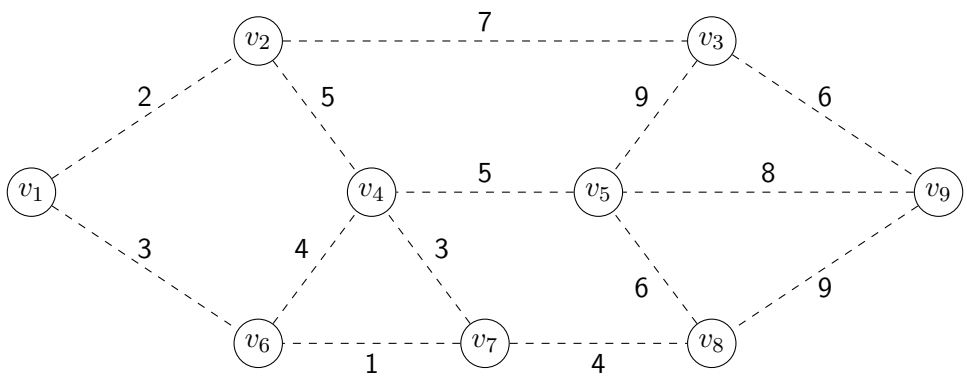
$$\boxed{T(m,n) \in O(n\log(n) + m\log(n))}$$

which at first glance looks like an improvement over our running time from before, but if G is the complete graph and $m = \frac{n(n-1)}{2}$, then this implementation takes $\Theta(n^2\log(n))$ time while the alternative implementation takes $\Theta(n^2)$ time.

Kurskal's Algorithm

Prim's algorithm is not the only algorithm that we have for finding a minimum spanning tree of a graph. Another popular algorithm is Kurskal's minimum spanning tree algorithm – we will not go into much detail on this algorithm but will demonstrate how it can be applied. The idea of the algorithm is the following:

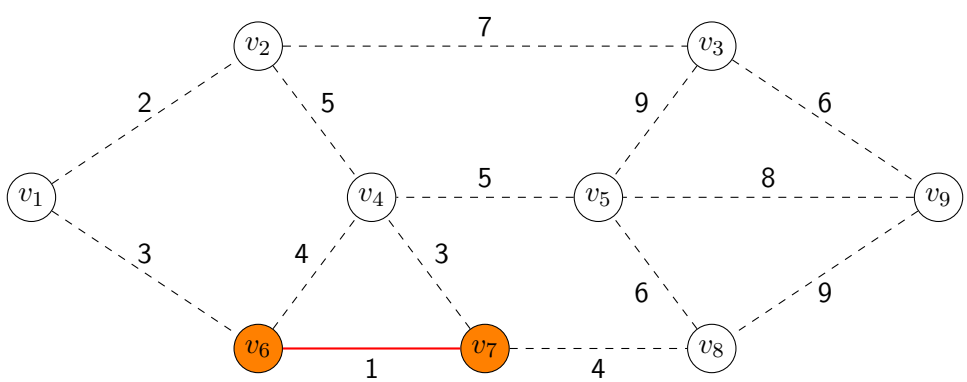
1. Create a set S_i for each vertex. That is, $S_1 = \{v_1\}$, $S_2 = \{v_2\}$, and so on.
2. Iterate over all of the edges of the graph and choose the edge with the smallest possible edge weight that would not create a cycle. Add that edge to the tree.
3. Union the two sets containing the vertices incident on the selected edge. If the two elements are in the same set, then adding the edge would create a cycle!
4. The algorithm terminates when all of the elements are in the same set.



First we create 9 disjoint sets

$$S_1 = \{v_1\}, S_2 = \{v_2\}, S_3 = \{v_3\}, S_4 = \{v_4\}, S_5 = \{v_5\}, S_6 = \{v_6\}, S_7 = \{v_7\}, S_8 = \{v_8\}, S_9 = \{v_9\}$$

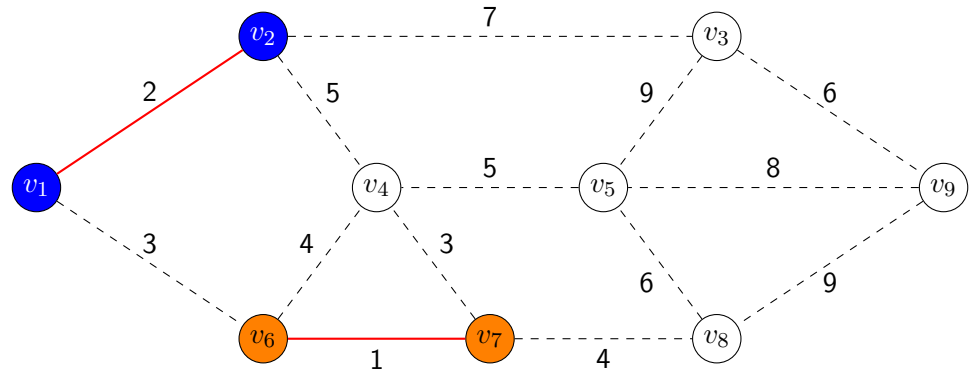
The minimum weight edge is (v_6, v_7) with an edge weight of 1. v_6 and v_7 are both in different sets, so we add (v_6, v_7) to the spanning tree and union their sets together.



The disjoint sets become

$$S_1 = \{v_1\}, S_2 = \{v_2\}, S_3 = \{v_3\}, S_4 = \{v_4\}, S_5 = \{v_5\}, S_6 = \{v_6, v_7\}, S_8 = \{v_8\}, S_9 = \{v_9\}$$

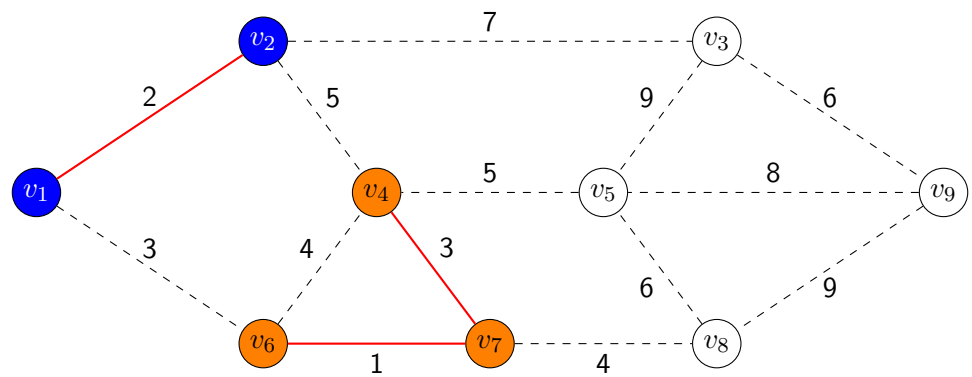
The minimum weight edge is (v_1, v_2) with an edge weight of 2. v_1 and v_2 are both in different sets, so we add (v_1, v_2) to the spanning tree and union their sets together.



The disjoint sets become

$$S_1 = \{v_1, v_2\}, S_3 = \{v_3\}, S_4 = \{v_4\}, S_5 = \{v_5\}, S_6 = \{v_6, v_7\}, S_8 = \{v_8\}, S_9 = \{v_9\}$$

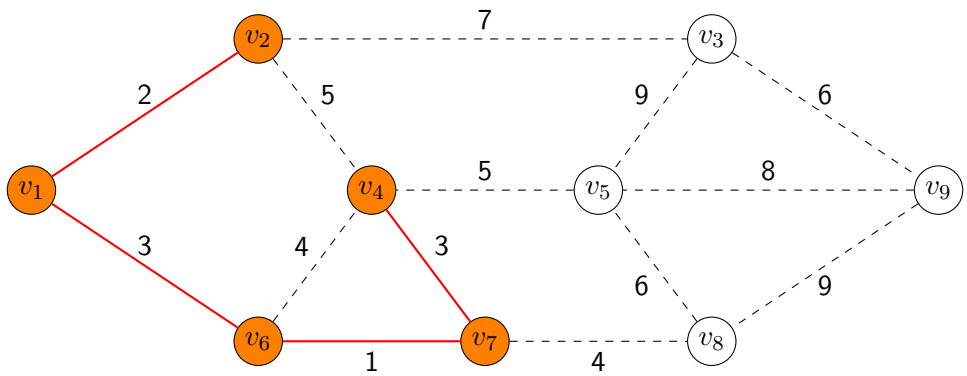
There are two edges with weight 3, so we must choose one of them to add. Let's suppose we add edge is (v_7, v_4). v_7 and v_4 are both in different sets, so we add (v_7, v_4) to the spanning tree and union their sets together.



The disjoint sets become

$$S_1 = \{v_1, v_2\}, S_3 = \{v_3\}, S_5 = \{v_5\}, S_6 = \{v_4, v_6, v_7\}, S_8 = \{v_8\}, S_9 = \{v_9\}$$

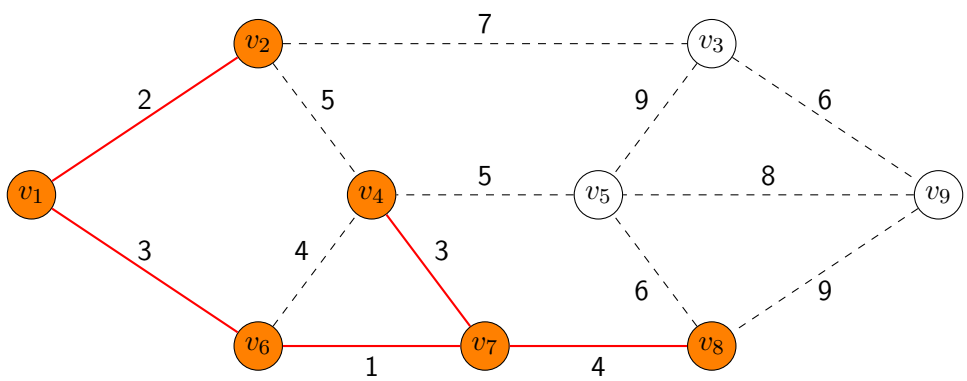
The minimum weight edge is (v_1, v_6) with an edge weight of 3. v_1 and v_6 are both in different sets, so we add (v_1, v_6) to the spanning tree and union their sets together.



The disjoint sets become

$$S_3 = \{v_3\}, S_5 = \{v_5\}, S_6 = \{v_1, v_2, v_4, v_6, v_7\}, S_8 = \{v_8\}, S_9 = \{v_9\}$$

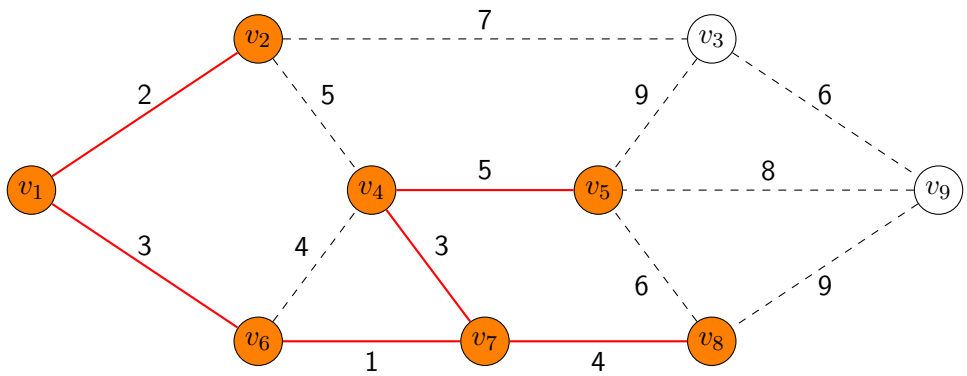
There are two edges with weight 4; edge (v_6, v_4) connects two vertices that are in the same set and therefore we will not add it to the spanning tree. We can also see that this would create a cycle of v_4, v_6, v_7. The other edge with weight 4 is (v_7, v_8) and v_7 and v_8 are in different sets, so we add (v_7, v_8) to the spanning tree and union their sets together.



The disjoint sets become

$$S_3 = \{v_3\}, S_5 = \{v_5\}, S_6 = \{v_1, v_2, v_4, v_6, v_7, v_8\}, S_9 = \{v_9\}$$

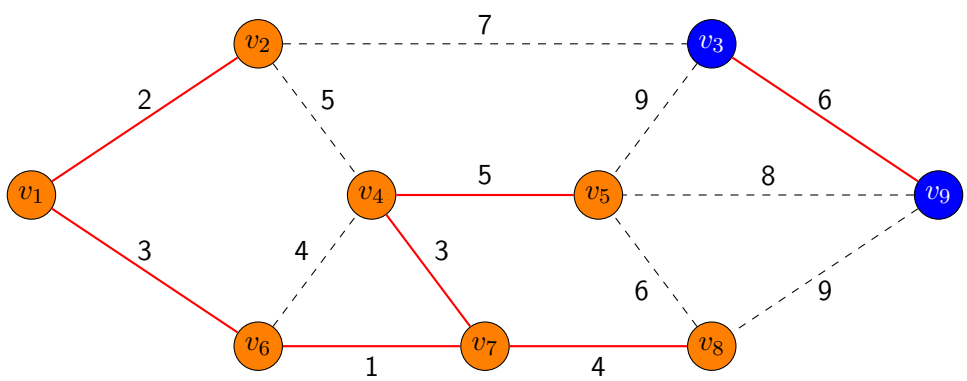
There are two edges with weight 5; edge (v_2, v_4) connects two vertices that are in the same set and therefore we will not add it to the spanning tree. We can also see that this would create a cycle of v_1, v_2, v_4, v_7, v_6. The other edge with weight 5 is (v_4, v_5) and v_4 and v_5 are in different sets, so we add (v_4, v_5) to the spanning tree and union their sets together.



The disjoint sets become

$$S_3 = \{v_3\}, S_6 = \{v_1, v_2, v_4, v_5, v_6, v_7, v_8\}, S_9 = \{v_9\}$$

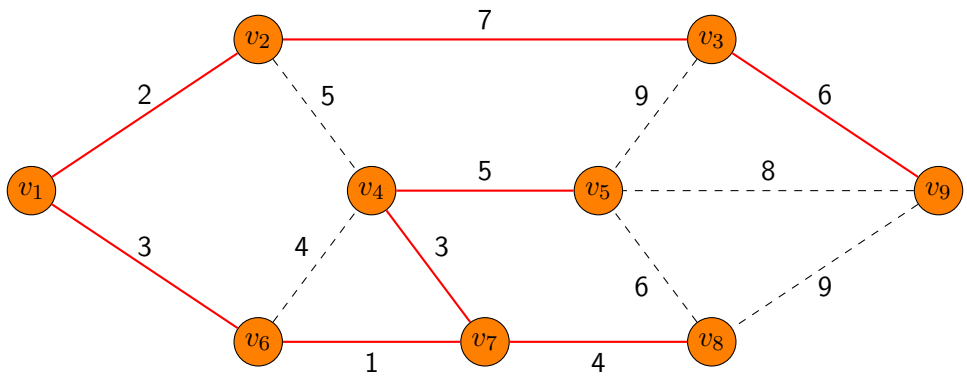
The minimum weight edge is (v_1, v_9) with an edge weight of 6 (note that (v_5, v_8) also has weight 6 but would cause a cycle!). v_3 and v_9 are both in different sets, so we add (v_3, v_9) to the spanning tree and union their sets together.



The disjoint sets become

$$S_3 = \{v_3, v_9\}, S_6 = \{v_1, v_2, v_4, v_5, v_6, v_7, v_8\}$$

The minimum weight edge is (v_2, v_3) with an edge weight of 7. v_2 and v_3 are both in different sets, so we add (v_2, v_3) to the spanning tree and union their sets together.



The disjoint sets become

$$S_6 = \{v_1, v_2, v_3, v_4, v_5, v_6, v_7, v_8, v_9\}$$

Therefore the algorithm terminates!